



Синтез и анализ на алгоритми за факултети ФКСТ, ФТК, ФЕТТ

проф. д-р Даниела Минковска
каб. 2324, кат. ПКТ, ФКСУ, ТУ-София
daniela@tu-sofia.bg



Лекция 10

**Методи за съставяне на алгоритми.
Рекурсия, метод „отгоре-надолу“
„разделяй и владей“, динамично
програмиране.**

“АЛГОРИТМИ + СТРУКТУРИ ОТ ДАННИ = ПРОГРАМИ”

Н. УИРТ



Методи за съставяне на алгоритми.

Формално съставянето на алгоритъма може да се раздели на две основни групи:

- “**поредово**” програмиране
- използване на **специални методи и подходи** с цел повишаване на ефективността на процеса на проектиране и реализация на сложни проекти



Методи за съставяне на алгоритми.

При втората група могат да се използват различни техники за съставяне на алгоритми, използвайки следните методи :

- Структурно програмиране
- Обектно-ориентирано програмиране
- Метод на частните цели
- Метод “Разделяй и владей”
- Метод “Връщане назад” (обратно проследяване или **backtracking**)
- Евристични алгоритми
- Рекурсия
- Динамично програмиране

Използването на който и да е от тези подходи в повечето случаи води до създаването на ефективни алгоритми, но не винаги има гаранция!



Структурно програмиране.

Цели на структурното програмиране:

- Да се осигури дисциплина в програмирането
- Да се подобри читаемостта на програмите
- Да се повиши ефективността на програмите
- Да се повиши надеждността на програмите
- Да се намали времетраенето и стойността на разработването на програмите



Структурно програмиране.

Основни принципи на структурното програмиране:

- Принцип на абстракциите
- Принцип на формалността
- Принцип “Разделяй и владей”
- Принцип на йерархичното подреждане

*Програмиране без **goto**!*

Произволно сложна програма може да бъде реализирана при използване на следния минимален базис:

Последователност

while (итеративна структура)

case (избор)

Всяка неструктурирана програма може да бъде структурирана допълнително.



Метод с връщане назад (*backtracking*).

Идея на метода *backtracking*:

Една не малка група от задачи в програмирането се свежда до намиране на отговор на някой от следните въпроси:

има ли начин ...?; колко начина съществуват ...?; да се намерят всички възможни начини ...

Последните два изискват пълно изчерпване на множеството от решения.

Започва се с **целта** (или решението) и се предприема **движение в обратната посока** – към началната постановка на задачата. След това пак от постановката към решението и т. н.



Метод с връщане назад (*backtracking*).

Идея на метода *backtracking*:

Методът се състои в следното:

1. На всяка стъпка се тръгва от едно частично k^{mo} решение
2. Последователно се разглеждат различните алтернативни варианти за продължаване на текущото k^{mo} решение
3. Ако е открит приемлив вариант, така намереното $k+1^{vo}$ частично решение става **базово** за продължаване на търсенето;
4. Ако всички алтернативи са изчерпани и **не** е намерен вариант за продължаване на решението, се прави връщане **назад към $k-1^{voto}$** частично решение за изграждане на нова хипотеза за k^{mo} решение и т.н.

За реализация на метода се използва рекурсия. Придвижването със стъпка напред се прави при рекурсивното обръщение, а връщането назад при затваряне на рекурсивното обръщение



Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

Пример 1: Нишката на Ариадна - една древногръцка легенда разказва, че на о-в Крит съществувал подземен лабиринт, в който живеел Минотавър. Всяка година жителите на острова трябвало да го омилостивяват като му изпращат по една девойка. Когато дошъл ред на Ариадна, нейният възлюбен Тезей решил да влезе вместо нея в лабиринта и да убие Минотавъра. Съобразителната Ариадна му дала едно кълбо. Докато търсел чудовището, Тезей развивал нишката от кълбото. Така след като се справил с Минотавъра, успял лесно да намери обратния път и да се измъкне от лабиринта.

Нашата задача е да помогнем на Тезей да открие Минотавъра и да маркира обратния път.



Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

Представяме лабиринта като двумерен масив *labyrinth* от символи, които имат следното значение:

<интервал> - проходима област (коридор); **-** - непроходима област; **M** – Минотавър; Със символа **.** ще маркираме изминатия път.

При движението ще спазваме следните правила:

1. Входът на лабиринта е клетката с координати **(1; 1)**.
2. Можем да се придвижваме **с една клетка** във вертикално или хоризонтално направление.
3. Можем да се придвижваме само към **проходима клетка ()** или към **Минотавъра (M)**.
4. **Не можем** да напускаме очертанията на лабиринта.

Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

След приключване на одисеята на екрана трябва да се изобрази целия лабиринт и следата, оставена от нишката (със символа **.**).

Условия по метода *backtracking*:

- Ако сме в клетката с Минотавъра, **задачата е решена!**
- Ако сме в проходима клетка, маркираме я с **.** и пробваме да продължим първо надясно, при неуспех нагоре, наляво и надолу, като проверяваме дали съответната клетка е проходима.
- Ако и четирите опита са неуспешни, връщаме се върху предходната клетка като заменяме символа **.** с **x**. Знакът **x** означава, че този вариант вече е проверен и няма отново да го пробваме. Клетките, означени с **x** означават че са непроходими клетки, и това ще ни спаси от зацикляне.

Преди извеждане на лабиринта ще заменим всички **СИМВОЛИ x с .**



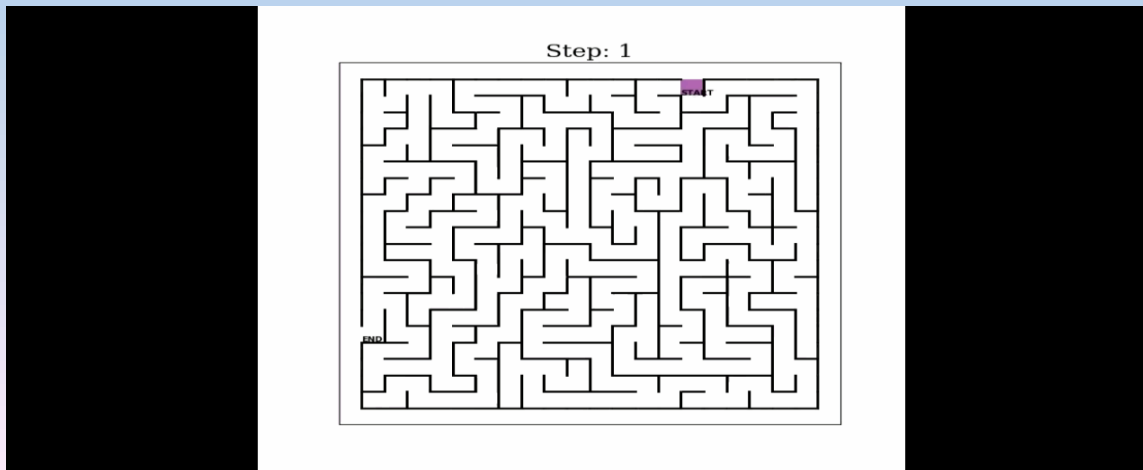
Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

Решение:

Дефинираме рекурсивна функция *FindPath(int x, int y, int found)*, където *x* и *y* са координатите на текущата клетка, а *found* ще има стойност 1 (истина), ако пътят до Минотавъра е открит и 0 (лъжа) в противен случай. В зависимост от нея ще решаваме дали да пробваме алтернативни варианти или не.

За да елиминираме частните случаи, предварително ще оградим лабиринта в рамка от непроходими клетки.





Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

Решение:

#include

const int MAX_ROWS=22,
MAX_COLS=22;

//От потребителя може да се дефинира
//област с максимум 20 реда и 20 колони

char labyrinth[MAX_ROWS][MAX_COLS]; *//Масивът, съхраняващ лабиринта*
//е глобален

void InitLab(int& rows, int& cols); *//Инициализира лабиринт с rows*
//реда и cols колони

void FindPath(int x, int y, int& found); *//Намира пътя от клетка (x,y) до*
//Минотавъра



Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

```
void PrintLab(int rows, int cols);    //Извежда лабиринта като при това  
                                     //заменясимвола 'x' с '.'  
  
int main()  
{  
  int rows, cols,  
  found = 0;                        //Първоначално приемаме, че път не е намерен  
  
  InitLab(rows, cols);  
  FindPath(1, 1, found);  
  cout << endl << "Нишката на Ариадна:n";  
  PrintLab(rows, cols);  
  return 0;
```



Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

Пример 2: Задача за джипа - Да се пресече пустиня с дължина 1000 мили с MIN разход на гориво. Обем на резервоара – 500 л, разходът на гориво е 1 л на 1 миля (за улесняване на сметките). В началния пункт има неограничени количества гориво, в пустинята – няма.

Очевидно е, че в пустинята трябва да се създадат складове. *Къде да ги направим и колко гориво да съхраняваме, за да пресечем пустинята с MIN разход на гориво?*

Ако имаме склад с **K** резервоара гориво, какво разстояние можем да изминем? *Къде трябва да се намира този склад, че да стигнем до края на пустинята?*

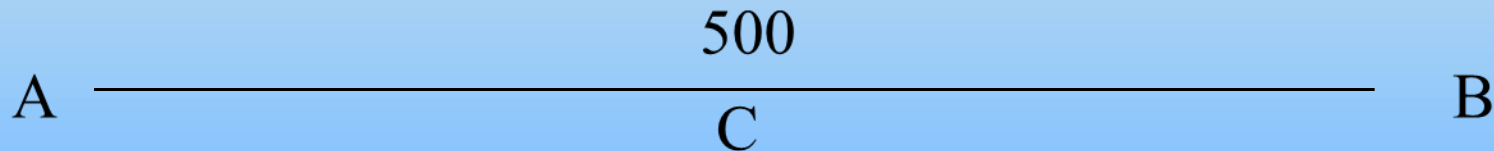
Ще търсим отговор на тази задача **за K=1, 2, 3, ... N**, докато **N** ще се окаже **достатъчно за 1000 мили**.



Метод с връщане назад (*backtracking*).

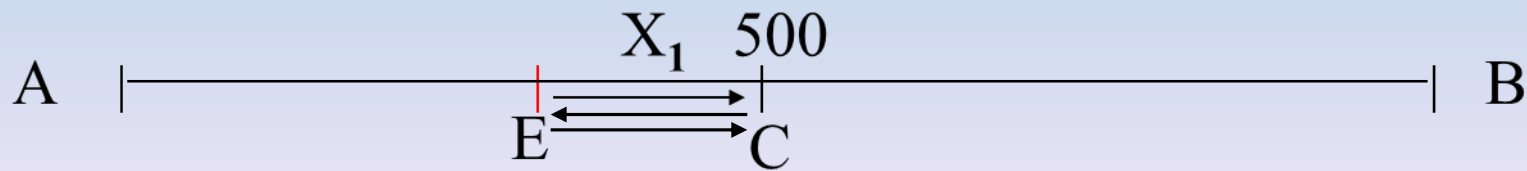
Примери за метода *backtracking*:

Нека **$K=1$** . Очевидно е, че могат да се изминат **500** мили:



C е най-отдалечената точка, започвайки от която джипът може да пресече пустинята, разполагайки с 500 л гориво. Т. е. в **C** трябва да осигурим склад с 1 резервоар (500 л) гориво. **Как?**

Нека **$K=2$** , т.е. в следващия склад ще има два пълни резервоара гориво (1000 л). **Къде да се разположи този склад, така че от него да се пренесат в C 500 л ($K=1$)?**



Т.е. каква е **Мах стойност** на **X_1** , че тръгвайки с 1000 л ($K=2$) от т. **(500- X_1)** могат да се занесат достатъчни количества в т. **C**.





Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

Един от начините е следния:

Зареждаме колата в т. **(500- X_1)** , стигаме до т. **С** (изминавайки X_1 мили), преливаме в склада цялото гориво с изключение на X_1 литра, необходими за връщане в т. **(500- X_1)**. Тук резервоарът вече е празен. Пълним го за втори път, изминаваме X_1 мили до т. **С**, зареждаме запазеното там гориво и тръгваме за т. **В** с пълен резервоар.

Общото разстояние, което се изминава, се състои от 3 отсечки по X_1 мили и СВ или в литри:

$$3 X_1 \text{ литра} + 500 \text{ литра} = 1000 \text{ литра}$$

Решението на уравнението е $X_1 = 500/3$.

Т.е. два резервоара бензин (или 1000 литра) ни позволяват да изминем:

$$D_2 = 500 + X_1 = 500 (1 + 1/3) \text{ мили}$$



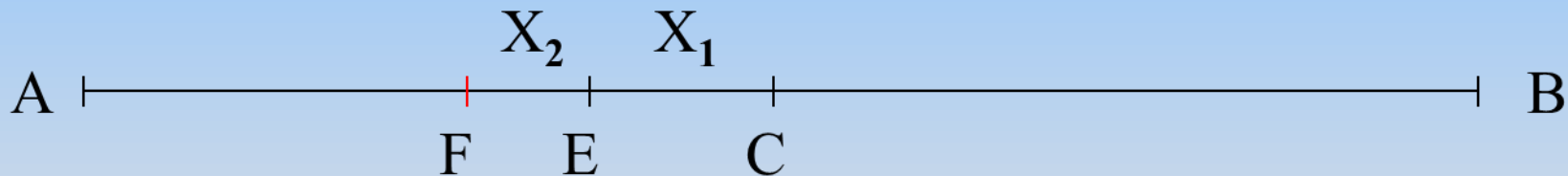
Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

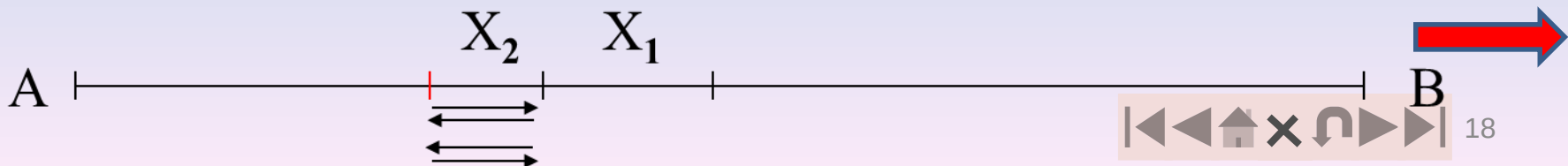
Постановката е следната:

като разполагаме с K резервоара гориво, ние се стремим да се придвижим колкото може повече назад от точката, в която се намират $K-1$ резервоара, т. е. търсим $\text{Max } X$.

От точка $K=3$ трябва да тръгнем с 1500 л гориво, за да доставим 1000 л в т. $(500 - X_1)$. Търсим $\text{Max } X_2$, което да ни осигури това условие:



Тръгваме с 1500 л гориво от т. $(500 - X_1 - X_2)$ (или т. F) и доставяме 1000 л в т. $(500 - X_1)$ (или т. E).





Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:

Това може да стане по следния начин:

тръгваме от т. $(500 - X_1 - X_2)$, стигаме до т. $(500 - X_1)$, преливаме горивото, като запазваме само необходимото X_2 количество. Връщаме се в т. $(500 - X_1 - X_2)$ или т. **F** с празен резервоар. Ако повторим процедурата, ние ще изразходваме $4X_2$ литра гориво и ще оставим $(1000 - 4X_2)$ литра в т. $(500 - X_1)$ или т. **E**. Сега в т. **F** остават само 500 л. Зареждаме ги и тръгваме за т. **E**, като изразходваме по пътя X_2 литра:

$$1500 - 5X_2 = 1000 \text{ (толкова остават в т. F)} \quad \text{т. е. } X_2 = 500/5$$

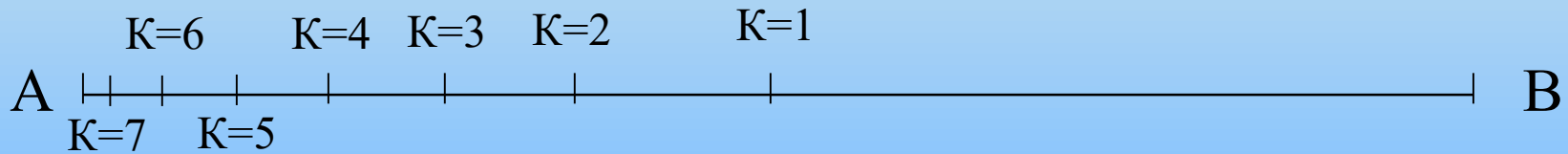
Или с три резервоара можем да се придвижим напред с:

$$D_3 = 500 + X_1 + X_2 = 500 (1 + 1/3 + 1/5) \text{ мили}$$



Метод с връщане назад (*backtracking*).

Примери за метода *backtracking*:



Ако продължим този процес, ще получим следната закономерност:

$$D_N = 500 (1 + 1/3 + 1/5 + \dots + 1/(2N-1))$$

Задачата се решава с $N = 7$



Рекурсия

Основи на рекурсията:

Рекурсията е метод в математиката и програмирането за дефиниране на функция или езикова конструкция чрез самата себе си.

Пример за рекурсивно дефиниране на функция е функцията факториел:

$f(n) = n! = n * (n-1) * (n-2) * (n-3) * \dots * 2 * 1$, като се приеме $0! = 1$

или

$$n! = f(n) = \begin{cases} 1, & n = 0 \\ n * f(n-1) & n > 0 \end{cases}$$

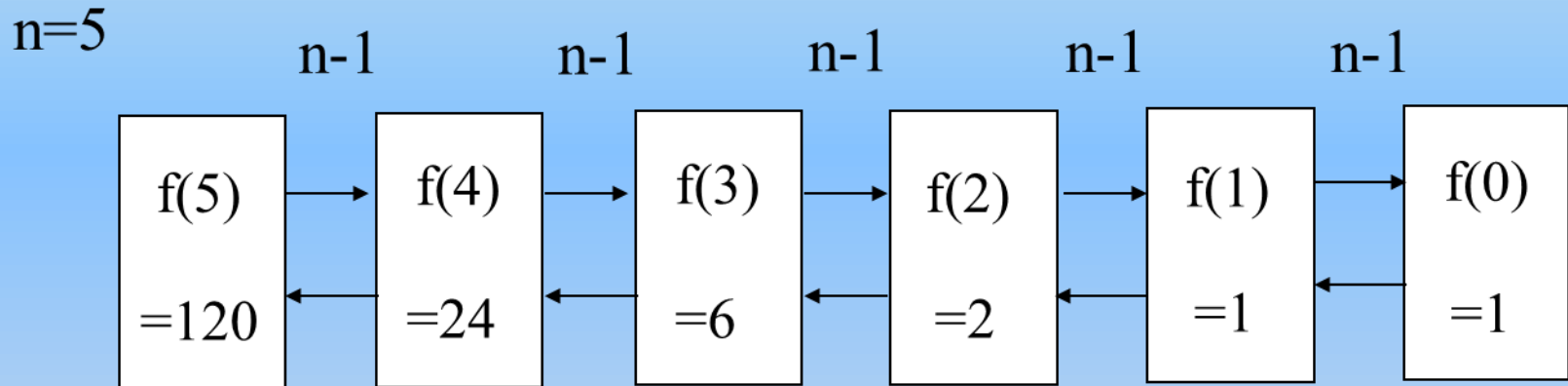
$$n! = n * (n-1) * (n-2) * \dots * 2 * 1$$



Рекурсия

Примери за рекурсия:

Пример1: намиране на $n!$



```
factorial( n ):
```

```
if n == 1:  
    return 1
```

```
else:
```

```
    return n * factorial(n-1):
```

```
        if n == 1:  
            return 1  
        else:
```

$factorial(n) =$



Рекурсия

Примери за рекурсия:

Рекурсивен вариант на функцията **F(n) (factorial(n))**:

```
#include <iostream.h>
long factorial (long n); //prototype
main()
{
    long k;
    cout<<"\nВъведете цяло число:";
    cin>>k;
    cout<<"\n"<<k<<"! = "<<factorial(k);
    return 0;
}
long factorial (long n)
{ if (n>1)
    return (n*factorial(n-1));
  else
    return(1);
}
```



Рекурсия

Примери за рекурсия:

Итеративен вариант на функцията **F(n) (factorial(n))**:

```
#include<iostream.h>
factor(int n);//prototype
main()
{
    int n;
    char c;
    do
    {
        cout<<"\nВъведете цяло число: ";
        cin>>n;
        factor(n);
        cout<<"\nДруга стойност (Y/N)? ";
        cin>>c;
    }
    while ((c!='N')&&(c!='n'));
    return 0; }
```

```
factor(int n)
{
    int rez=1;
    for(int i=1;i<=n;i++)
        rez*=i;
    cout<<"\nn!="<<rez;
    return 0;
}
```




Рекурсия

Примери за рекурсия:

Пример 2: Алгоритъм на Евклид на намиране на най-големия общ делител на две положителни числа

$$NGOD(M, N) = \begin{cases} NGOD(N, M), & N > M \\ M & N = 0 \\ NGOD(N, M \% N), & N \leq M \end{cases}$$



Рекурсия

Примери за рекурсия:

Рекурсивен вариант:

```
#include <iostream.h>
#include <conio.h>
int NGOD (int m,int n); //Prototype
main()
{
    int m, n;
    char r;
    do
    {
        do
        {
            cout<<"\nm=";
            cin>>m;
            cout<<"\nn=";
            cin>>n;
        }
    }
```

```
while ((n<0) || (m<0));
cout<<"\nNGOD("<<m<<"
"<<n<<")="<<NGOD(m,n);
    cout<<"\nNext arg
(y/n):";
    cin>>r;
}
while ((r=='y') || (r=='Y'));
return 0;
}
```



Рекурсия

Примери за рекурсия:

Рекурсивен вариант:

```
int NGOD(int m, int n)
{
    if (n>m)
        return(NGOD(n,m));
    else
    {
        if (n==0)
            return(m);
        else
            return(NGOD(n,m%n));
    }
}
```

По-ефективна реализация:

```
int NGOD (int m, int n)
{ if (n>m)
    return NGOD(n,m);
  else return n?NGOD(n,m%n):m;
}
```

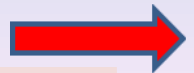


Рекурсия

Примери за рекурсия:

Итеративна реализация:

```
int NGOD (int n, int m)
{
    int k, nn, mm;
    if (n < m)
        {nn = n; mm = m;}
    else
        {nn = m; mm = n;}
    while (nn)
    {
        k = nn;
        nn = mm % nn;
        mm = k;
    }
    return mm;
}
```





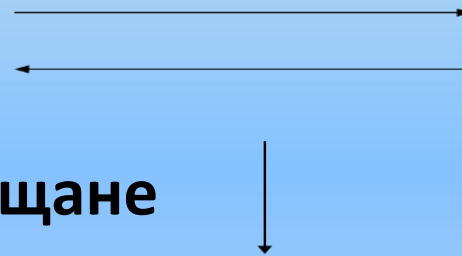
Рекурсия

Форми на рекурсивни функции:

Рекурсивните обръщения могат да се разглеждат като движение:

напред/назад

или рекурсивно спускане/завръщане



Всяка рекурсивна функция се състои от определен брой оператори (**S** - *тяло или действие*), конструкция **if** (*условие*) (задължително!), и едно или няколко **рекурсивни обръщения**.



Рекурсия

Форми на рекурсивни функции:

Форма 1:

S се изпълнява **преди** рекурсивно обръщение (т. е. при рекурсивно спускане):

```
int Rec (параметри)
{
    S;
    if (условие)
        Rec (параметри);
}
```



Рекурсия

Форми на рекурсивни функции:

Форма 2:

S се изпълнява **след** рекурсивно обръщение (т. е. при рекурсивно връщане):

```
int Rec (параметри)
{
    if (условие)
        Rec (параметри);
    S;
}
```



Рекурсия

Форми на рекурсивни функции:

Форма 3:

Действието се извършва както **преди**, така и **след** рекурсивното **обръщение** (при спускане и при връщане):

```
int Rec (параметри)
{
    S1;
        if (условие)
            Rec (параметри);
    S2;
}
```

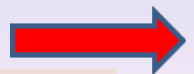



Рекурсия

Примери за рекурсивни функции:

Пример 3: Рекурсивна програма за формиране на аритметична прогресия: $a_i = a_{i-1} + b$, $a_i \leq c$

```
#include <iostream.h>
void prog (int n);           //prototype
int a, b, c;
main()
{
    cout <<"\nВъведете a:";  cin>>a;      // Например a=1
    cout <<"\nВъведете b:";  cin>>b;      //          b=2
    cout <<"\nВъведете c:";  cin>>c;      //          c=7
    prog(a);
    return 0; }
void prog (int n)
{
    cout<<n<<" ";           // 1 3 5 7
    if (n+b<=c)
        prog(n+b);
    cout<<n<<" ";           // 7 5 3 1
}
```





Рекурсия

Примери за рекурсивни функции:

Пример 4: Задача за разполагане на царици върху шахматна дъска (или задача за 8-те царици)

Задачата е пример за използване на метода “на пробите и грешките”, а също така метода на обратно търсене (или връщане назад, или *backtracking*). През 1850 върху тази задача е работил Гаус, но така и не успял да получи пълното ѝ решение, т.к подобни задачи обикновено нямат аналитическо решение, те изискват голям обем неприятни сметки, търпение и точност. Затова подобни задачи се решават предимно от компютри.

Условието на задачата е: Да се състави програма, която намира всички възможни начини за разполагане на N царици (максимално възможната стойност на N е 8) върху шахматна дъска по такъв начин, че никоя царица да не заплашва друга царица.

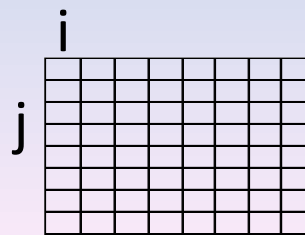


Рекурсия

Примери за рекурсивни функции:

Естественят **подход** за намиране на решение на задачата е **рекурсивен**: ако сме разположили удачно **i** царици, **проблемът се свежда до намиране на задоволително положение на $i+1$ -вата царица**. Задачата е решена, когато **$i=N$** . Остава да изведем намереното решение.

Известно е, че царицата заплашва всяка фигура в реда, колоната и диагонала, в които се намира, т.е. **всеки ред и всяка колона може да съдържа само една царица**. Цариците можем да представим с номерата на съответните клони (**i**). Тогава изборът на позиция за i -та царица се ограничава в i -та колона и се свежда до определяне на номера j на реда, в който тя може да се разположи в тази колона:





Рекурсия

Примери за рекурсивни функции:

Полуформалното описание на алгоритъма за разполагане на i -та царица е:

```
PLACE(int i)
{
    for (j=0; j< n; j++)
        { if допустимо i-та царица в j-я ред
            {
                Регистрация на i-та царица;
                if (i<n) {PLACE(i+1)}
                else PRINT;
                Премахване на регистрацията на i-та царица;
            }
        }
    } {PLACE}
```



Рекурсия

Примери за рекурсивни функции:

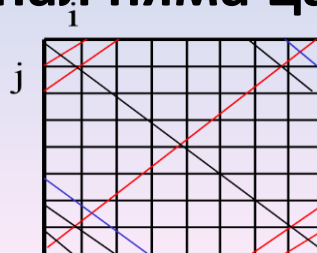
За да разположим **i-та** царица задоволително, трябва да знаем в кои редове и диагонали са поставени царици до момента. Как да представим тази информация?

Ако разгледаме шахматната дъска, то можем да открием следната закономерност: за “**червените**” диагонали сумата от координатите им $i+j$ е константа: **$i+j=\text{const}$; $0 \leq i+j \leq 2(N-1)$**

За “**сините**” диагонали: **$i-j=\text{const}$; $1-N \leq i-j \leq N-1$**

Така се дефинират, които да съхраняват нужната ни информация: **три булеви масива**

- **$a[j]=1$** , ако в j ред няма царица, и 0, ако има;
- **$b[i+j]=1$** , ако в този диагонал няма царица, и 0, ако има;
- **$c[i-j]=1$** , ако в този диагонал няма царица, и 0, ако има;





Рекурсия

Примери за рекурсивни функции:

Проверката на тези три масива ще ни позволява да определим, дали i -та царица може да бъде поставена в j -я ред. При положителен отговор i -та царица се поставя в клетката с координати (i,j) , след което тази позиция се регистрира – съответните елементи на масивите a , b и c се установяват в 0.

Продължаваме рекурсивно да търсим място на следващата, $i+1$ -та царица. Ако това не е възможно при $i < N$, връщаме се обратно към i -та царица и се опитваме да я преместим в друга клетка с надеждата, че това ще доведе до друго решение. Непосредствено преди връщането е необходимо да премахнем регистрацията на текущата позиция на i -та царица, т.е. да присвоим на съответните елементи на масивите a, b и c стойност 1.



Рекурсия

Примери за рекурсивни функции:

Ядрото на програмата е рекурсивната **функция PLACE**. В тялото на функцията е организиран цикъл за изчерпване на всички възможни варианти за разполагане на дадената царица.

```
#include <iostream.h>
```

```
void print();
```

```
void true_false(int i, int j, int k);
```

```
void place(int i);
```

```
const int N=8;
```

```
int a[N]={1,1,1,1,1,1,1,1};
```

```
int b[2*N-1]={1,1,1,1,1,1,1,1,1,1,1,1,1,1,1};
```

```
int c[2*N-1]={1,1,1,1,1,1,1,1,1,1,1,1,1,1,1};
```

```
int pos[N]={0,0,0,0,0,0,0,0};
```



Рекурсия

Примери за рекурсивни функции:

```
main()  
{  
    place(1);  
    return 0;  
}
```

```
void print()  
{  
    for (int m=0; m<N; m++)  
        cout<< " " <<pos[m];  
    cout<<endl;  
}
```

```
void true_false(int i, int j, int k)  
{  
    a[j-1]=k;  
    b[i+j-2]=k;  
    c[i-j+7]=k; }  
}
```




Рекурсия

Примери за рекурсивни функции:

```
void place(int i)
{
    int j;
    for (j=1; j<=N;j++)
    {
        if((a[j-1]==1)&&(b[i+j-2]==1)&&(c[i-j+7]==1))
        {
            pos[i-1]=j;
            true_false(i,j,0);
            if (i<N)
                place(i+1);
            else
                print();
            true_false(i,j,1);
        }
    }
}
```



Рекурсия

Примери за рекурсивни функции:

Пример 5: Друг класически пример за използване на рекурсия е формирането на **редицата на Фибоначи**. Числата на Фибоначи представляват безкрайна поредица от цели числа, в която елементът с индекс **N ($N > 2$)** е равен на сумата на двата непосредствено предхождащи го елемента- с индекси **$N-1$ и $N-2$** .

Първите два елемента на поредицата са **0 и 1**:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89,...

$$FIB(N) = \begin{cases} 0, & N = 1 \\ 1, & N = 2 \\ FIB(N-1) + FIB(N-2), & N > 2 \end{cases}$$



Рекурсия

Примери за рекурсивни функции:

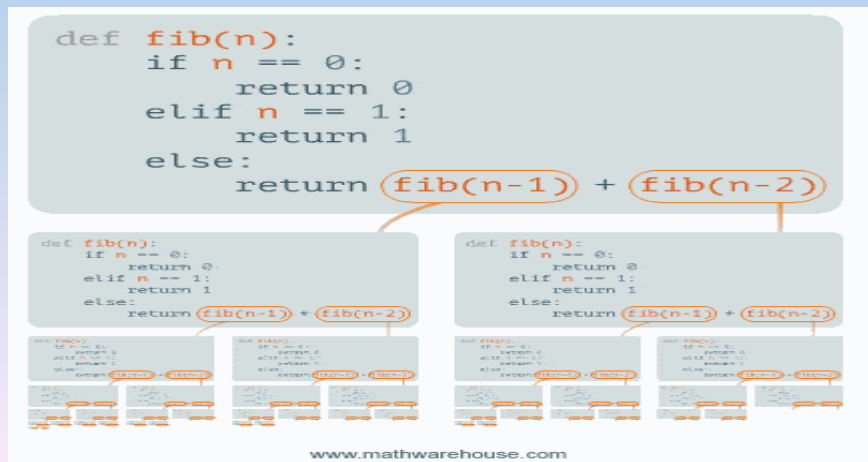
Рекурсивна реализация на програма за изчисляване числата на Фибоначи:

```
#include<iostream.h>
#include<conio.h>
int fib (int i); //prototype
main()
{
    int nn;
    cout<<"Въведете броя на числата (не повече от 24): ";
    cin>>nn;
    for (int j=0; j<nn; j++)
        cout<<fib(j)<<" ";

    return 0;
}
```

```
int fib (int i)
{
    if (i<1)
        return 0;
    if (i==1)
        return 1;
    return fib(i-1)+fib(i-2);
}
```

Реализацията е не ефективна!





Рекурсия

Примери за рекурсивни функции:

Итеративна реализация на програма за изчисляване числата на Фибоначи:

```
#include<iostream.h>
#include<conio.h>
const n=24;// max за int
main()
{
    int f[n];
    int nn;
    cout<<"Въведете броя на числата (не повече от 24: ";
    cin>>nn;
    f[0]=0;
    f[1]=1;
```

```
    for (int j=2; j<nn; j++)
        f[j]=f[j-1]+f[j-2];
    cout<<"\nЧислата на Фибоначи
    са:\n ";
    for (j=0; j<nn; j++)
        cout<<f[j]<<" ";
    return 0;
}
```



Рекурсия

Ефективност на рекурсията:

При всяко рекурсивно обръщение към подпрограма (функция) се **заделят клетки от ОП** за параметрите и за локалните променливи.

Възможно е **системният стек да се окаже недостатъчен** за изпълнението на програмата, в която има много рекурсивни обръщания (*т.е. дълбочината на рекурсията е голяма*). Подобен проблем възниква също така и **при неправилно рекурсивно обръщение – стекът се препълва**.

В случая на голяма дълбочина на рекурсията и невъзможност програмата да се реализира по този начин необходимо е да се премине към другия подход – итеративен.

По-принцип **рекурсивните програми са по-бавни и по-трудно се тестват**. Използването на рекурсия **изисква по-голям обем ОП**.



Рекурсия

Ефективност на рекурсията:

От друга страна, рекурсията води до повишаване на ефективността на програмисткия труд. Много са задачите (и ние видяхме някои от тях), които се програмират трудно и бавно без рекурсия и просто и бързо – с рекурсия.

Рекурсивните програми са **по-четливи, по-ясни и разбираеми** от съответните нерекурсивни програми.

Затова трудно можем да намерим средно-сложна програма на някой от известните езици за програмиране, в която да няма рекурсивни обръщения. Но трябва да се има предвид, че **не всички езици допускат рекурсия (например, Фортран, Бейсик, Кобол)**. Но и чрез тях може да се моделира механизма на рекурсията.

Съвременните компютри по-ефективно се справят с нерекурсивните програми, отколкото с рекурсивните.



Рекурсия

Ефективност на рекурсията:

Съвременните транслатори не са в състояние автоматично да заменят рекурсията с цикъл и така да подобрят изпълнението на рекурсивните програми. Но в бъдеще може да има такива компютри и транслатори, които ефективно ще изпълняват рекурсивните програми. Т.е. трябва да се използват всички възможности за използване на рекурсия в програмирането.

Кога НЕ ТРЯБВА да прилагаме рекурсия:

- когато тя е **неефективна** в степен, по-голяма за допустимата в конкретния случай (напр. бързодействието пада и пречи на работата, или дълбочината на рекурсията е такава, че процесът на обръщанията е нереализуем, т.е. препълване на системния стек);
- когато **нерекурсивното решение е по-просто, по-естествено и много по-ефективно от рекурсивното** (като задачите за събиране и умножение на цели числа).



Метод „разделяй и владей“

Идея на метод „разделяй и владей“:

Идеята е да се раздели сложната задача на няколко по-прости, които се атакуват по-лесно поотделно и чието решаване позволява лесно конструиране на решение на изходната задача.

Прилагането на метода протича на три стъпки:

- Първо се извършва разбиване на изходната задача на няколко подзадачи, най-често две (*разделяй*).
- Следва решаване на всяка от тях поотделно.
- На третата стъпка, въз основа на решенията на подзадачите, се конструира решение на изходната задача (*владей*).



Метод „разделяй и владей“

Идея на метод „разделяй и владей“:

Разбира се, принципът не е универсален и не при всяка задача прилагането му е възможно и подходящо. Причините, поради които методът не може да се приложи, могат да бъдат най-разнообразни.

Например:

- задачата може да не позволява подходящо разбиване на подзадачи, които могат да се решават поотделно.
- възможно е решаването на отделните подзадачи да не облекчава съществено решаването на изходната задача или дори да няма нищо общо с него.

Макар идеята на метода да е проста, прилагането му, както и определянето кога може да бъде приложен, е много или малко изкуство, което най-лесно се усвоява на основата на конкретни примери.



Метод „разделяй и владей“

Примери за метод „разделяй и владей“:

Примери за метода:

В лекция 7:

- Бързо сортиране (quicksort) – на Хоар

В лекция 8:

- Сортиране чрез сливане (mergesort)
- Алгоритъм на Бетчер(четно-нечетно сортиране):



Динамично програмиране

Принцип на динамичното програмиране:

Метод, при който се извършва **запомняне на междинните резултати** от решенията на вече решени подзадачи с цел **избягване на повторни пресмятания**, се нарича **динамично програмиране** (*dynamic programming*) или **динамично оптимиране**.

Основното предназначение на този метод е **решаването на различни оптимизационни задачи**.



Динамично програмиране

Примери за динамичното програмиране:

Пример 1: За да разгледаме метода, нека да използваме известния вече пример за *изчисляване на числата на Фибоначи* (рекурсивна и итеративна реализация). На практика рекурсивната реализация изисква много излишни обръщания към функцията. За да се изчисли n -то число всеки път се изчисляват $n-1$ -то и $n-2$ -то числа, което е изключително **неефективно**. Броят на извършваните рекурсивни обръщания е ненужно голям. Алгоритъмът е експоненциален и няма да работи при големи n . При динамичната реализация изчислените вече числа се съхраняват в статичен масив ***known[]*** (или друг външен за рекурсивната функция масив) и се използват при изчисляване на следващите стойности.

Алгоритъмът има **полиномиална сложност**.





Динамично програмиране

Примери за динамичното програмиране:

Динамична реализация на програма за изчисляване на числата на Фибоначи:

```
#include<iostream.h>
#include<conio.h>
int fib(int i); //prototype
main()
{
    int nn;
    cout<<"nn=";
    cin>>nn;
    for (int j=0; j<nn; j++)
        cout<<fib(j)<<" ";
    return 0;
}
```

```
int fib (int i)
{
    static int knownf[n];
    if (knownf[i]!=0)
        return knownf[i];
    int t=i;
    if (i<0)
        return 0;
    if (i>1)
        t=fib(i-1)+fib(i-2);
    return knownf[i]=t;
}
```



Динамично програмиране

Примери за динамичното програмиране:

Пример 2: Още една класическа оптимизационна задача, която се решава чрез динамично оптимиране:

ЗАДАЧА ЗА РАНИЦАТА - Дадена е раница с вместимост M килограма и N предмета, всеки от които се характеризира с две числа: m_i (тегло) и c_i (стойност). Да се избере такова множество предмети, чиято сумарна стойност е максимална, а сумата от теглата им не надвишава M . Числата M , N , m_i , и c_i са естествени числа ($1 \leq i \leq N$).

Задачата се свежда до намиране на максимума на сумата:

$$\sum_{i=1}^N x_i c_i \quad \sum_{i=1}^N x_i m_i \leq M$$

$$c_i > 0, m_i > 0, x_i \in \{0,1\}, i = 1, 2, \dots, N$$





Динамично програмиране

Примери за динамичното програмиране:

Решение: Ще дефинираме рекурентна целева функция $F(i)$, представляваща решение за раница с вместимост i .

Тогава:

$$F(i) = \begin{cases} 0 & i = 0 \\ \max_{j=1,2,\dots,N; m_j \leq i} [c_i + F(i - m_j)] & i > 0 \end{cases}$$

$i = 0$

$i > 0$

i	v	w
1	5	4
2	4	3
3	3	2
4	2	1

Capacity=6

		w						
		0	1	2	3	4	5	6
i	0							
	1							
	2							
	3							
	4							

Рекурентната формула трябва да бъде реализирана като рекурсивна функция. За да се извърши динамично оптимиране, за всяка $F(i)$, ще се поддържа множество $s[i]$, съдържащо конкретните предмети, чието вземане води до тази минимална стойност. Освен това $s[i]$ ще ни предпази от повторното включване на един и същи предмет.



Динамично програмиране

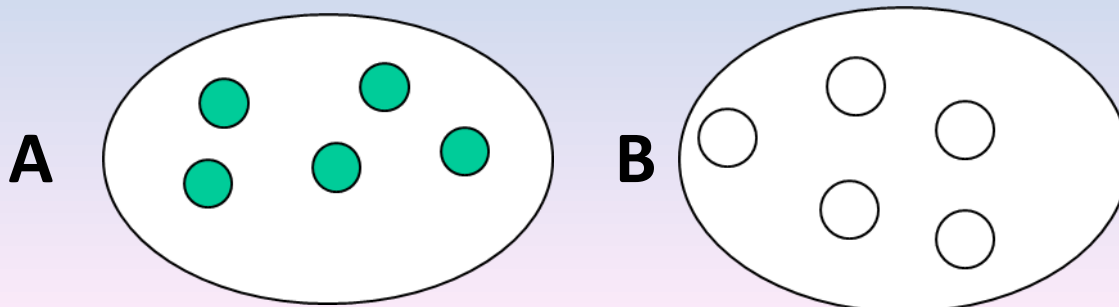
Примери за динамичното програмиране:

Пример 3: Задача за стабилните бракове.

Дадени са две непресичащите множества A и B с по n елемента. Необходимо е **да се намери множество двойки $[a, b]$ от n елемента, такива, че $a \in A$, $b \in B$ да отговарят на определени условия** (критерии).

Критериите могат да бъдат най-различни. Единият от тях се нарича **“Правило на стабилните бракове”**.

Нека A да е множество, съдържащо n мъже, а B – множество, съдържащо n жени. Всички те желаят да сключат брак и си имат определени предпочитания.





Динамично програмиране

Примери за динамичното програмиране:

Необходимо е да се състави алгоритъм, който да формира **п двойки** на базата на тези предпочитания. Ако двойките са съставени така, че съществуват мъж и жена, които не образуват по между си двойка (*т.е. участват в различни двойки*), но предпочитат един друг, то множеството бракове се нарича **“нестабилно”**.

Ако не съществува такава ситуация (*т.е. такава “неправилна” двойка*), множеството се нарича **“стабилно”**.

Тази задача има значително по-общ характер и се решава много често: класиране по ВУЗ-ове и специалности след кандидат-студентски изпити, разпределение на студенти по групи за свободно избираеми дисциплини и т.н.



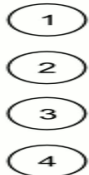
Динамично програмиране

Примери за динамичното програмиране:

Решението ще се търси чрез последователно получаване на двойки елементи от двете множества, до изчерпване на множествата. Нека да намерим **всички** стабилни комбинации, като за целта ще използваме подхода от задачата за 8-те царици. Нека **try(m)** е алгоритъм за **търсене на партньорка** за **мъжа m** и нека търсенето става съгласно списъка на предпочитания на този МЪЖ.

Round : 1

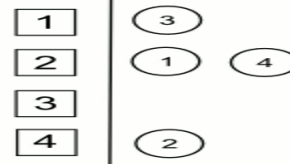
Proposors



Acceptors



Proposal pool



• 1-4 propose, as none are currently tentatively attached

Preferences

□ → ○

Acceptor Table

1	1	3	2	4
2	3	4	1	2
3	4	2	3	1
4	3	2	1	4

○ → □

Proposor Table

1	2	1	3	4
2	4	1	2	3
3	1	3	2	4
4	2	3	1	4





Динамично програмиране

Примери за динамичното програмиране:

```
int    man, woman;           //1..n
try(int man)
int    rank;                  //1..n;
{    for (r=1; r<= n; r++)
    {    От списъка на предпочитания на m-я мъж да
        се избере    r-та жена;
        if (изборът е приемлив)
        {
            фиксиране на брака;
            if (m<>n) try(succ(m));
            else да се запише стабилното множество;
            да се отмени брака;
        }
    }
}
```